

STEMKit: A client-side toolkit for computational chemistry and scientific data analysis

Olanrewaju M. Daramola^{a,1,*}

^a*Independent Researcher*

Abstract

STEMKit is a suite of 18 browser-based tools for computational chemistry and scientific data analysis, built on `@stemkit/core`, a JavaScript library of parsing and numerical routines with no DOM dependency. All computation runs inside the user's own browser: nothing is transmitted, no account is required and nothing is installed, so the tools stay usable for unpublished or confidential data. The same modules run under Node.js, so an interactive analysis can be captured as a version-pinned script. The 17 domain modules carry 1128 tests validated against SciPy, NumPy and physical invariants rather than against the implementation itself. That validation exposed four defects that had been altering reported results, among them p -values floored at zero and the mass of haem iron taken as fluorine's. All four are corrected and covered by regression tests.

Keywords: Computational chemistry, Molecular dynamics, Client-side computation, GROMACS, Reproducibility, Data privacy

*Corresponding author.

Email address: lanrelangmuir@gmail.com (Olanrewaju M. Daramola)

¹ORCID: 0009-0006-3327-2047

Required Metadata

Current code version

Nr.	Code metadata description	Metadata
C1	Current code version	v0.1.1
C2	Permanent link to code/repository used for this code version	https://github.com/LD-Shell/stemkit Archived release v0.1.1: https://doi.org/10.5281/zenodo.21544868 (the concept DOI 10.5281/zenodo.21543112 resolves to the current release)
C3	Legal code license	MIT License
C4	Code versioning system used	git
C5	Software code languages, tools and services used	JavaScript (ECMAScript 2020 modules), HTML5, CSS3; Node.js; Jest; Tailwind CSS; GitHub Actions
C6	Compilation requirements, operating environments and dependencies	Browser tools: any current browser supporting ECMAScript modules and the FileReader API; no installation and no build step. Library: Node.js ≥ 18 ; <code>npm install</code> installs the development dependencies (Jest, Tailwind CSS) only. There are no runtime dependencies: <code>jStat</code> , <code>Papa Parse</code> , <code>regression.js</code> and <code>bibtex-parse-js</code> are vendored in the repository
C7	If available, link to developer documentation/manual	https://github.com/LD-Shell/stemkit#readme ; per-module API documentation in <code>src/-core/README.md</code> ; hosted tools at https://stemkit.net
C8	Support email for questions	lanrelangmuir@gmail.com (issue tracker: https://github.com/LD-Shell/stemkit/issues)

Table 1: Code metadata (mandatory)

1. Motivation and significance

The software landscape of computational chemistry is well served at its extremes. Simulation engines such as GROMACS [1] and LAMMPS [2] are mature, extensively validated, and actively maintained; analysis frameworks such as MDAnalysis [4] provide comprehensive programmatic access to trajectory data. Between these poles lies a substantial volume of routine work that neither category serves comfortably.

Consider the interval between completing a simulation and reporting it. A researcher must inspect the RMSD to confirm equilibration, extract a radius of gyration, convert a structure between the nanometre convention of GROMACS and the Å convention of the Protein Data Bank, compare replicate measurements statistically, digitise a figure from a paper for comparison, abbreviate journal titles in a reference list, and compose a scheduler script requesting an appropriate resource allocation. None of these operations is intellectually demanding. All of them are error-prone when performed by hand, and all of them are performed repeatedly.

1.1. The friction problem

Three classes of solution exist, and each imposes a characteristic cost.

Local scripting. A Python environment with MDAnalysis, NumPy and SciPy addresses every task listed above and is the correct instrument for sustained analytical work. Its cost is environmental. Managed institutional workstations frequently prohibit package installation; conda environments drift; and a script written six months ago may no longer execute against the current interpreter. For a researcher whose primary expertise lies in chemistry rather than software engineering, this cost is not negligible.

Web services. Server-side tools eliminate installation entirely but require data transmission. For unpublished results, material subject to a confidentiality agreement, or work under embargo, this is frequently prohibited outright. Even where permitted, the transfer creates a record whose retention policy the researcher does not control.

Desktop applications. Graphical applications avoid both problems but introduce a third: they are difficult to script. An analysis performed by clicking cannot readily be re-executed, version-controlled, or subjected to continuous integration. In practice such an analysis is difficult to reproduce, even when every step was recorded at the time.

1.2. *The client-side position*

The browser is a capable computational environment. **FileReader** grants access to local files without transmission, and the engines are fast enough for the volumes involved: the parser sustains roughly 7×10^5 rows per second, reading a two-million-row, 53 MB `.xvg` in 2.9 s, and parses a million-atom PDB and returns its centre of mass in 1.3 s. The ceiling is memory rather than arithmetic, since **FileReader** materialises the file as a string; a few hundred megabytes is practical, beyond which streaming would be required. ECMAScript modules make it possible to organise that code properly, instead of accumulating scripts.

STEMKit is built around these constraints. No server participates in the computation, so there is nothing to which data could be transmitted and users are not asked to take a privacy policy on trust. Every asset is vendored in the repository: there is no content-delivery network, no analytics and no telemetry, so not even a request log is generated elsewhere, and a local clone runs with the network disconnected. The reproducibility property derives from a separate architectural decision: the numerical core is extracted into modules with no dependency on the browser, so the identical code that executes behind the interface also executes under Node.js.

1.3. *Experimental setting*

The software is used in two ways sharing one implementation. Interactively, the user opens a tool page at <https://stemkit.net> (or a local clone), selects a local file and reads the result in the page; the file is read through **FileReader** and never uploaded. Programmatically, the same routines are imported from `@stemkit/core` in a Node.js script, the path taken once an analysis is to be fixed, version-pinned and re-run. Every tool that produces a figure additionally emits a standalone matplotlib script that regenerates that figure from the original data file, so an interactive result can be reproduced offline without the browser.

Related work is cited where the corresponding functionality is described in Section 2: GROMACS [1], LAMMPS [2] and PLUMED [3] for the file formats, MDAnalysis [4] as the reference framework, 3Dmol.js [17], whose atom-selection syntax the `selection` module mirrors, SciPy [5] and NumPy [6] for validation, the primary statistical literature [8, 10, 12, 13, 14, 15] for the implemented tests, CODATA 2018 [7] for conversion factors, and standard practice for the incomplete beta and gamma functions [16].

2. Software description

The 18 research tools are the subject of this paper. Three further browser utilities (a Pomodoro timer, a decision matrix and a kinetics sandbox) are distributed from the same repository as general workflow aids; they are not part of the scholarly contribution and are not described here, though they share the presentation layer and so appear in the page counts below.

2.1. Software architecture

The original implementation followed the pattern common to browser tooling: each of the 21 tool pages consisted of an HTML document paired with a single JavaScript file combining event handling, rendering and computation. This arrangement is expedient during development and progressively obstructive thereafter. Computational logic entangled with DOM manipulation cannot be tested without a browser, cannot be reused across tools, and cannot be invoked from a script.

The refactoring applied the *strangler fig* pattern: for each tool the mathematical and parsing logic was extracted into a pure module beneath `src/core/`, a test suite written against it, and the original file reduced to DOM wiring importing the new functions. The interface is unchanged; the computation beneath it is now independently addressable. Figure 1 shows the resulting layering and Table 2 enumerates the modules.

Four third-party libraries are vendored in the repository: jStat, Papa Parse, regression.js and bibtex-parse.js. Each ships as a Universal Module Definition (UMD) bundle, which detects its host at run time and either assigns `module.exports` under CommonJS or attaches a global to `window`. Neither path is reachable from a plain ECMAScript module: a UMD file has no `export` statements, so a direct import yields no bindings, while `import { createRequire } from 'module'` is a hard *resolution* failure in a browser, occurring before any code runs and so uncatchable. The dynamic `await import()` does reject catchably, but injection was preferred because the bundles must be available synchronously. Rather than branch on the execution environment inside every module, which would render the core untestable in one of its two targets, the library declares the libraries it requires and each host registers them at initialisation (Section 2.3). Modules request their dependencies lazily, at the point of use rather than at import time, so importing any module never fails merely because an unrelated library has not yet been registered.

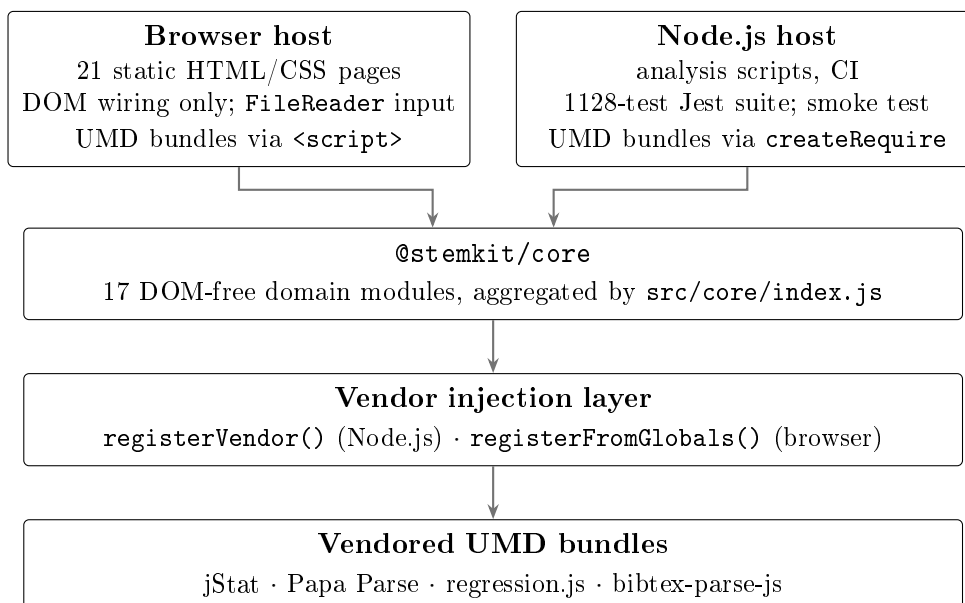


Figure 1: Architecture of STEMKit. Both hosts execute the identical core modules; the only environment-specific code is the registration call that supplies the vendored libraries.

Table 2: Modules of `@stemkit/core`. Vendored dependencies are supplied by injection; modules marked *none* operate without any third-party code.

Module	Domain	Tests	Dependency
bibtex	Reference deduplication, sanitising	102	bibtex-parse-js
structure	Molecular geometry, PDB/GRO/XYZ	100	none
statistics	Inferential statistics	84	jStat
curve-fitting	Least-squares regression	77	regression.js
xvg-parser	GROMACS/PLUMED trajectory data	77	none
units	Physical unit conversion	73	none
plumed	PLUMED input generation	72	none
selection	Atom selection, spatial queries	67	none
slurm	SLURM job scripts, resource checks	61	none
latex	Table generation and escaping	59	none
error-bars	Group summaries, error bars	57	jStat
journals	Whole-title journal abbreviation	53	none
data-cleaning	Tabular transformation	51	Papa Parse
outliers	Anomaly detection	51	jStat
scheduler	Scheduler directives and launchers	51	none
digitizer	Figure digitisation	49	none
iso4	ISO 4 word-level abbreviation (LTWA)	44	none
Total		1128	

The test suite comprises 1128 tests across the 17 domain modules, with 94.1% statement and 97.6% line coverage of `src/core/`. Its governing principle is that numerical results are validated against *independent* references rather than against the implementation under test, since a test written from the same source as the code confirms only internal consistency. Reference values were obtained from SciPy [5] for t-tests, ANOVA, correlation, non-parametric tests, quantiles and multiple-comparison correction; from NumPy [6] for regression coefficients and descriptive statistics; from `scipy.constants` for every unit conversion factor; and from physical invariants for structural geometry, specifically the molecular weight and centre of mass of water, the orthonormality of rotation matrices, the preservation of interatomic distances under rotation, and round-trip fidelity through every supported file format. An end-to-end smoke test additionally exercises one representative path through fifteen of the seventeen modules against a real installation, detecting the class of failure that unit tests cannot: a broken aggregate export, a misconfigured module type declaration, or a vendored bundle that fails to load.

2.2. Software functionalities

Table 3 maps each tool to the modules it calls; the subsections below describe the numerics grouped by domain.

2.2.1. Trajectory and collective-variable data

GROMACS writes analysis output in the Grace format, in which `@` directives carry the plot title, axis labels and per-series legends alongside the numeric records. A parser that skips all non-numeric lines discards precisely the meta-data identifying which column holds which observable; `xvg-parser` recovers it with the matrix. Malformed records are counted and reported rather than silently dropped, so a truncated trajectory is visible instead of quietly shortening the analysis. Two further details of the format are handled explicitly: the Grace multi-set separator `&` is treated as a delimiter rather than parsed as data, and records containing `NaN` or `Infinity` are rejected whole, since one non-finite entry would otherwise contaminate every downstream statistic.

PLUMED [3] writes `COLVAR` files whose `#!` headers follow a different convention but share Grace’s comment character, so the same parser handles both without modification. An explicit test verifies this rather than leaving it to assumption.

Table 3: The 18 research tools and the core modules behind them. Three further pages (Pomodoro timer, decision matrix, kinetics sandbox) are workflow aids and are excluded.

Tool	Module(s)	Purpose
XVG visualiser	xvg-parser	Plot GROMACS/PLUMED series with recovered axis metadata
Structure inspector	structure, selection	Geometry, mass breakdown, atom selection, 3D view
Coordinate manipulator	structure	Translate, rotate, re-box; PDB/GRO/XYZ interconversion
MD workflow generator	slurm, scheduler, plumed	Batch scripts for GROMACS and LAMMPS on four schedulers; PLUMED input
Statistics calculator	statistics	t-tests, ANOVA, correlation, non-parametric tests, assumption checks
Error-bar generator	error-bars	Group summaries, error bars, significance annotation
Outlier detector	outliers	Tukey fences, modified <i>Z</i> -score, Grubbs' test
Curve fitter	curve-fitting	Least-squares fits with model-adequacy warnings
Plot digitiser	digitizer	Pixel-to-data recovery with log-axis and uncertainty handling
Plot builder	statistics	Publication figures with matplotlib script export
Data cleaner	data-cleaning	Delimited-text repair, type inference, reshaping
Scientific converter	units	64 units in ten categories, CODATA-sourced
BibTeX sanitiser	bibtex	Field normalisation and escaping
BibTeX deduplicator	bibtex	Union-find over normalised DOIs and titles
DOI to BibTeX	bibtex	Crossref lookup with field filtering
Journal abbreviator	journals, iso4	Whole-title dictionary, then ISO 4 word-level fallback
LaTeX table builder	latex	LaTeX and Markdown tables with correct escaping
Equation editor	latex	Formula entry and LaTeX output

2.2.2. Structure files and molecular geometry

The `structure` module parses the three coordinate formats in routine use for classical simulation: PDB and GROMACS `.gro`, both fixed-column, and whitespace-delimited XYZ. Fixed-column parsing is essential rather than pedantic: a residue name may legitimately be blank, and a whitespace-splitting parser would silently shift every subsequent field on that line. Unit handling is explicit throughout: PDB and XYZ express coordinates in Å, `.gro` uses nanometres, and every parse result carries its unit. Silently mixing the two is the most direct route to a structure that is wrong by a factor of ten.

Assigning an element to each atom is a prerequisite for any mass-weighted quantity, and PDB atom names are ambiguous by construction: `CA` denotes the α -carbon in a protein residue and calcium in an ion record. The resolution implemented here proceeds in five stages.

1. An explicit element column, where present, takes priority.
2. A leading digit is a hydrogen-count prefix (`1HB`, `2HG1`) and is stripped before the symbol is read.
3. An unambiguous two-letter metal (`FE`, `ZN`, `SE`) is recognised wherever it appears, including within a host residue such as haem or selenomethionine.
4. Otherwise, a two-letter prefix whose second character is uppercase is a PDB remoteness indicator, and only the first letter is the element: `CA` in a protein residue is $C\alpha$.
5. As a final fallback, a two-letter symbol is trusted when the residue shares its name, the convention for monatomic ions.

Stages 2 and 3 correct defects present in the original implementation (Section 2.2.5). The resolution is validated against 40 atom names drawn from real structures, covering protein backbone and side-chain atoms, nucleic acid atoms, water, monatomic ions and metal centres. On this basis the module computes the geometric centroid, the mass-weighted centre of mass, the axis-aligned bounding box, and the radius of gyration

$$R_g = \sqrt{\frac{\sum_i m_i |\mathbf{r}_i - \mathbf{R}|^2}{\sum_i m_i}}, \quad \mathbf{R} = \frac{\sum_i m_i \mathbf{r}_i}{\sum_i m_i}, \quad (1)$$

together with rigid-body rotation about an arbitrary pivot using intrinsic Z – Y – X Euler angles. Velocities present in a `.gro` file rotate with the frame but are never translated, so a round trip through rotation and export preserves the kinetic state.

2.2.3. Statistics, outliers and curve fitting

The `statistics` module implements Welch’s [8] and Student’s t -tests, the paired t -test, one-way ANOVA, Pearson correlation, the Mann–Whitney U test and the Wilcoxon signed-rank test. Welch’s test is the default for independent samples, as it does not assume equal variances and loses negligible power when they are in fact equal. Every parametric test reports an effect size and, where standard, a confidence interval. Assumptions are checked and surfaced rather than presumed: normality by the D’Agostino–Pearson K^2 omnibus test [10], homogeneity of variance by the median-centred Brown–Forsythe variant of Levene’s test [12], chosen for its greater robustness to non-normality. Where an assumption fails, a non-parametric alternative is recommended. Table 4 maps each computed quantity to its numerical method and validation reference.

Table 4: Numerical methods and validation references.

Quantity	Method	Validated against
Student / Welch t, p	Regularised incomplete beta $I_x(a, b)$	<code>scipy.stats.ttest_ind</code>
Welch–Satterthwaite ν	Closed form	SciPy, hand calculation
One-way ANOVA F, p	Sum-of-squares decomposition; I_x tail	<code>scipy.stats.f_oneway</code>
Pearson r , CI	Fisher z transform	<code>scipy.stats.pearsonr</code>
Mann–Whitney U	Tie-corrected normal approximation	<code>scipy.stats.mannwhitneyu</code>
Wilcoxon W	Signed-rank, zeros discarded	<code>scipy.stats.wilcoxon</code>
D’Agostino–Pearson K^2	$\sqrt{b_1}$ [9] and b_2 [11] transforms	<code>scipy.stats.normaltest</code>
Levene W	Brown–Forsythe (median-centred)	<code>scipy.stats.levene</code>
Quantiles	Linear interpolation (type 7)	<code>numpy.percentile</code>
Multiple comparisons	Holm–Bonferroni [13]	Independent implementation
Grubbs G	Inverted t critical value	<code>scipy.stats.t.ppf</code>
Modified Z -score	Median/MAD, 0.6745 scaling [14]	<code>numpy</code>
Least-squares fitting	Normal equations (regression.js)	<code>numpy.polyfit</code>

Upper-tail probabilities are computed through the complementary form of the incomplete beta and gamma functions rather than as $1 - F(x)$. The two

expressions are algebraically identical; numerically they are not. Once $F(x)$ rounds to unity in double precision, the subtraction cancels completely and the result is exactly zero. The consequence is not academic. For the one-way ANOVA used in the test suite ($F = 377.545$ on 2 and 21 degrees of freedom), the naive form returns $p = 0$, whereas

$$p = I_{\frac{d_2}{d_2+d_1F}}\left(\frac{d_2}{2}, \frac{d_1}{2}\right) = 3.461\,174\,738\,289\,425\,5 \times 10^{-17}, \quad (2)$$

reproducing `scipy.stats.f.sf` to full double precision. A continuous test statistic cannot produce $p = 0$, so that value in published output indicates a numerical fault rather than an unusually strong effect. The same treatment is applied to the Student t , χ^2 and normal tails. Beyond $|z| = 8$ the vendored `erfc` underflows and a documented asymptotic expansion takes over, accurate to 3.5×10^{-6} relative at the switchover and 1.6×10^{-10} by $z = 30$; at $z = 38$ it returns 5.8×10^{-316} , where `2*scipy.stats.norm.sf(38)` has already underflowed. Its own floor lies near $z = 38.5$, and that bound rather than any general principle is what limits how small a reported p can be.

One limitation of `curve-fitting` is documented rather than fixed, and it applies to two of the three transformed models rather than to all of them. The vendored `regression.js` library fits the *power* model by regressing $\ln y$ on $\ln x$ unweighted, so the residuals minimised are those in logarithmic space and small y values carry more influence than a direct fit would give them. The *exponential* model also works in log space but weights each point by y , which is the standard correction for log-transform bias and keeps the result closer to a direct non-linear fit. The *logarithmic* model carries none of this bias: it regresses y on $\ln x$, so only the predictor is transformed and the residuals minimised are those in y , exactly as for any model linear in its parameters. Where the bias does apply it is material. On an exactly doubling series the distinction vanishes, since the log-space residuals are identically zero and every least-squares variant recovers $\ln 2$; it appears only once the data are perturbed. For the series (1, 2.0), (2, 4.1), (3, 8.2), (4, 16.1), (5, 32.3) the weighted fit returns a growth rate of 0.690 216 against 0.693 167 for unweighted log-space least squares, with $\ln 2 = 0.693 147$ as the generating value: a difference of 0.4 % in the rate constant. Neither value is incorrect, since they answer different questions, but the gap matters when a rate constant is reported. `fitCurve` therefore exposes a `linearised` flag, true for the exponential and power models only, so callers may surface the distinction; users requiring publication-grade non-linear fits are directed to Levenberg–Marquardt optimisation on untransformed data. A dataset containing points a model cannot transform is refused with a message naming the requirement (exponential needs every $y > 0$, power every $x > 0$ and $y > 0$, logarithmic

only $x > 0$) rather than having the offending points silently dropped.

2.2.4. Digitisation, unit conversion and authoring utilities

Recovering values from a published figure requires a two-point affine calibration per axis. On a logarithmic axis the interpolation must be done in logarithmic space,

$$v(p) = 10^{\log_{10} v_1 + \frac{(p - p_1)(\log_{10} v_2 - \log_{10} v_1)}{p_2 - p_1}}, \quad (3)$$

because a decade occupies a constant pixel distance. Treating a logarithmic axis linearly is the commonest error in figure digitisation, and it yields values that are plausible rather than obviously wrong. The module also reports the data-space uncertainty of a one-pixel positioning error, since a digitised point’s precision is set by the figure’s resolution, not by the decimal places the export prints.

Sixty-four units across ten categories are each defined by their ratio to a category base unit, so conversion reduces to $x_{\text{to}} = x_{\text{from}} \cdot f_{\text{to}}/f_{\text{from}}$. The table is therefore linear rather than quadratic in the number of units, and every entry is independently checkable against its source: factors are CODATA 2018 values [7], validated against `scipy.constants` from SciPy 1.17.1, which ships the 2022 adjustment; the two agree to a relative 1×10^{-10} , far below the tolerance of any conversion the tools perform. Temperature is handled separately, being affine rather than multiplicative.

Two aspects of scheduler scripts dominate wasted allocation. First, the resource model differs by engine: GROMACS is threaded and wants one MPI rank per node with many CPUs per task, whereas LAMMPS [2] is MPI-parallel and wants many ranks per node. The wrong shape either idles most of a node or oversubscribes it. Second, and less widely appreciated, the SLURM `--mem` directive in an array job specifies memory *per task*, not per job: a request of 32 GB across a hundred-task array asks the scheduler for 3.2 TB concurrently. The generator computes the true in-flight total and reports it. Generated scripts resume from the last checkpoint rather than silently restarting from $t = 0$, a failure mode that consumes an entire allocation while appearing to succeed. Warnings are returned as structured objects, leaving presentation to the caller.

A single resource request is emitted for any of four schedulers, SLURM, PBS Professional/OpenPBS, LSF and Grid Engine, because the same allocation is expressed by different directives, environment variables and launch commands on each; values that are site-dependent, among them the queue name,

the name of the parallel environment and the resource under which GPUs are requested, are marked as such in the generated script rather than guessed.

The remaining modules support manuscript preparation: LaTeX and Markdown table generation with correct escaping, ISO-4 journal title abbreviation, and BibTeX deduplication by union-find over normalised DOIs and titles.

2.2.5. Defects exposed by extraction

Separating computation from presentation subjected the numerical code to independent verification for the first time, and four defects were identified that had propagated into reported output. First, skewness and kurtosis are defined against the *population* standard deviation,

$$G_1 = \frac{\sqrt{n(n-1)}}{n-2} \cdot \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma_{\text{pop}}} \right)^3, \quad \sigma_{\text{pop}} = \sqrt{\frac{1}{n} \sum_i (x_i - \bar{x})^2}, \quad (4)$$

whereas the original implementation used the Bessel-corrected sample standard deviation, deflating skewness by a factor of $((n-1)/n)^{3/2}$, approximately 15% at $n = 10$. Because both moments feed the D’Agostino–Pearson statistic, every normality p -value the tool reported was affected. Second, all upper-tail probabilities were computed by subtraction and floored at zero for strong effects. Third, haem iron was assigned the mass of fluorine (18.998 Da rather than 55.845 Da), selenomethionine selenium the mass of sulfur, and numeric-prefixed hydrogens no mass at all, so molecular weights and centres of mass computed for metalloproteins were correspondingly wrong. Fourth, the D’Agostino–Pearson transformations are defined on the biased moment ratios $\sqrt{b_1} = m_3/m_2^{3/2}$ and $b_2 = m_4/m_2^2$, but the sample-size-adjusted G_1 of Eq. 4 was passed into the skewness transform instead. The adjustment inflates $\sqrt{b_1}$ by exactly $\sqrt{n(n-1)}/(n-2)$: a factor of 1.247 at $n = 8$, 1.186 at $n = 10$ and 1.039 at $n = 40$. The error therefore vanishes on symmetric data and grows with skew. Because that factor exceeds unity for every n , the inflated statistic is always the larger and the reported p always the smaller, so the bias is systematically anti-conservative rather than merely noisy: the verdict at $\alpha = 0.05$ can only ever move from retaining normality to rejecting it, which is precisely the branch that redirects a user from Welch’s test to Mann–Whitney. This defect is the instructive one, because an independent reference was already in place and already disagreed. The residual had been absorbed by loosening the assertion to three decimal places and blaming SciPy’s kurtosis transform; that transform is identical, and the fixture was nearly symmetric ($g_1 = -0.0063$), so the inflation multiplied approximately zero. The reference value was correct and the implementation

was not. K^2 now reproduces `scipy.stats.normaltest` to twelve significant figures across $8 \leq n \leq 1000$, the full range over which the S_U approximation is tabulated [10] and the provenance of the module’s $n \geq 8$ guard, while `skewness()` continues to report G_1 , the appropriate statistic to publish and the one matching `scipy.stats.skew(bias=False)`.

Each defect is corrected and covered by a regression test. Users who generated figures with earlier versions of these tools should re-check any reported skewness, normality p -value, or metalloprotein mass.

2.3. Sample code snippets analysis

Listing 1 contains the whole of the environment-specific code in the library. Each host registers the vendored bundles once, after which the identical modules run unchanged in both.

```
// Node.js: the vendored bundles are loaded through
createRequire.
import { createRequire } from 'module';
import { registerVendor } from '@stemkit/core';

const require = createRequire(import.meta.url);
registerVendor({ jStat: require('./js/dependencies/jstat.min.js') });

// Browser: the UMD <script> tags have already installed
their globals.
import { registerFromGlobals } from './src/core/index.js';
registerFromGlobals();
```

Listing 1: Registration of the vendored libraries under Node.js and in the browser.

3. Illustrative examples

The example below is the post-simulation triage that motivated the toolkit: confirm that a production run has equilibrated, characterise the final structure, test a replicate comparison and queue a continuation. Every step is available interactively as a tool page; the scripted form is shown because it is the reproducible one.

Listing 2 reads a GROMACS RMSD series. The parser returns the numeric matrix together with the title and per-series legends recovered from the `@` directives, so each column is reported under its own observable name rather than by index.

```

import { parseXvg, extractColumn, columnStats } from '@stemkit/core';
import { readFileSync } from 'fs';

const { matrix, headers, title } = parseXvg(readFileSync('rmsd.xvg', 'utf8'));
// title -> "RMSD & Radius of Gyration"
// headers -> ["Time (ps)", "Backbone RMSD", "Rg"]

headers.slice(1).forEach((name, i) => {
  const s = columnStats(extractColumn(matrix, i + 1));
  console.log(`${name}: ${s.mean.toFixed(4)} +/- ${s.std.toFixed(4)} nm`);
});

```

Listing 2: Parsing a GROMACS RMSD trajectory and summarising each series.

Listing 3 takes the final coordinates and computes the mass-weighted centre of mass and radius of gyration. Because the input is a `.gro` file its coordinates are in nanometres, and the conversion to Å is requested explicitly rather than assumed.

```

import { parseStructure, centreOfMass, radiusOfGyration,
  convert }
  from '@stemkit/core';

const frame = parseStructure(readFileSync('final.gro', 'utf8'), 'gro');
const com = centreOfMass(frame.atoms); // nm, mass-weighted
const rg = radiusOfGyration(frame.atoms, com); // nm

console.log(`Rg = ${convert(rg, 'nm', 'angstrom').toFixed(2)} Å`);

```

Listing 3: Geometry of the final frame, with an explicit unit conversion.

Listing 4 compares two sets of replicate measurements. The returned object carries the assumption checks alongside the test result, so a violated assumption is visible at the point of use rather than discovered later; here a failed normality check redirects the comparison to Mann–Whitney.

```

import { welchTest, mannWhitneyU } from '@stemkit/core';

const r = welchTest(wildType, mutant);
// r.p, r.effectSize (Hedges' g), r.ci, r.assumptions

if (!r.assumptions.normality.passed) {

```

```

    console.warn(r.assumptions.normality.recommendation);
    console.log(mannWhitneyU(wildType, mutant));
  } else {
    console.log('Welch t(${r.df.toFixed(1)}) = ${r.t.toFixed(3)}, p = ${r.p}');
  }

```

Listing 4: A replicate comparison with the assumption checks surfaced.

Finally, Listing 5 generates the submission script for the continuation run. The engine argument selects the GROMACS resource shape (one rank per node, many CPUs per task) and the returned warnings report the true in-flight memory total for the array as well as any request the scheduler is likely to reject.

```

import { generateScript } from '@stemkit/core';

const { script, warnings } = generateScript({
  engine: 'gromacs', jobName: 'prod_md', partition: 'gpu',
  nodes: 1, gpus: 1, cpusPerTask: 16,
  walltime: '24:00:00', memory: '32G',
  modules: ['gcc/11.3', 'cuda/12.1', 'gromacs/2023.3'],
  tpr: 'md.tpr', deffnm: 'md', maxh: 23.5
});

warnings.forEach(w => console.warn(`${w.level} ${w.message}`));

```

Listing 5: Generating a GROMACS submission script with resource warnings.

The same four steps through the browser produce identical numbers, since they call the same functions. Where a step produces a figure, the tool also emits a matplotlib script that regenerates it from the original data.

4. Impact

The architecture’s most direct effect is on work that server-based tools cannot do at all. Where upload is prohibited, the relevant comparison is not with a faster tool but with doing the calculation by hand.

For research questions already being pursued, the gain is in reproducibility and in the reliability of the numbers. An analysis first explored by clicking can be captured as a script and version-pinned without being rewritten in another language, which is the step at which reproducibility is most often lost, and the emitted matplotlib scripts close the same loop for figures. Every reported statistic is checked against SciPy or NumPy rather than against

the author’s own expectations, and the suite is meant to be run by users evaluating the software: a library whose numerical claims cannot be checked offers reproducibility only in principle.

The defects found during extraction (Section 2.2.5) carry a wider lesson, and are the most useful outcome of this work. All four had survived in software that ran without error and produced plausible numbers. Three would not have been caught by testing the code against expectations derived from the same source; each surfaced only when results were compared with an independent implementation. The fourth is the more uncomfortable case, because the independent comparison had already been made and had already disagreed: the tolerance was widened until the test passed and the residual was explained away. An independent reference is necessary but not sufficient. A disagreement with one must be treated as a defect until shown otherwise, and a loosened tolerance is where that demonstration usually fails to happen. A 15% error in a reported skewness is not visible by inspection, and neither is a p -value floored at zero, which looks like a very strong result. Authors of small, rarely reviewed in-house analysis code would be well advised to check their numerics against an independent implementation before reporting output.

In daily practice, the change is the removal of a setup step from tasks that previously carried one. Checking whether an RMSD has converged, or whether a replicate difference survives an assumption check, becomes an operation performed at the moment the question arises rather than one deferred until an environment is available. The tools are also usable in teaching and in shared computing environments where installation is not an option, since a URL is the only prerequisite.

STEMKit is MIT-licensed, hosted at <https://stemkit.net> and archived on Zenodo [18]. The first archived release is contemporaneous with this manuscript, so adoption data (downloads, unique users, citing publications) does not yet exist. The arguments above are accordingly claims about what the design makes possible, not evidence of uptake, and should be read as such. The software is not used in a commercial setting and has not led to a spin-off company; the client-side design does, however, make it directly applicable to industrial research groups whose data-handling policies preclude uploading results to a third-party service.

5. Conclusions

STEMKit is a suite of 18 browser-based tools for computational chemistry built on a tested, dependency-injected JavaScript core of 17 modules. The architecture addresses three constraints simultaneously that existing tooling addresses only in pairs: it requires no installation, transmits no data and remains scriptable.

Both properties are structural rather than contractual: computation occurs entirely within the browser, and the modules behind the interface are the ones that run under Node.js. The 1128-test suite validates every numerical result against an independent reference, and the four defects that this strategy exposed (deflated standardised moments, tail probabilities floored at zero, misassigned elements in metalloproteins, and an adjusted moment fed into a transform defined on the unadjusted one) are documented, corrected and covered by regression tests. Future work will extend format coverage in the trajectory and structure modules and replace the two log-space fits with a Levenberg–Marquardt implementation on untransformed data.

Declaration of generative AI use

Claude (Anthropic) was used in preparing this manuscript: restructuring an earlier draft into the present template, correcting reported figures against the test suite, drafting and revising prose, and reviewing the numerical claims. The software itself, the numerical methods, and the verification of every figure and statement reported here are the author’s own. The tool is not an author and bears no responsibility for the content.

Acknowledgements

I thank the maintainers of jStat, Papa Parse, regression.js, Plotly, KaTeX and 3Dmol.js, whose libraries STEMKit builds upon.

Declaration of competing interest

The author declares no competing financial or non-financial interests.

References

- [1] M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, and E. Lindahl, GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers, *SoftwareX* **1–2**, 19–25 (2015). <https://doi.org/10.1016/j.softx.2015.06.001>
- [2] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, and S. J. Plimpton, LAMMPS — a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, *Comput. Phys. Commun.* **271**, 108171 (2022). <https://doi.org/10.1016/j.cpc.2021.108171>
- [3] G. A. Tribello, M. Bonomi, D. Branduardi, C. Camilloni, and G. Bussi, PLUMED 2: New feathers for an old bird, *Comput. Phys. Commun.* **185**(2), 604–613 (2014). <https://doi.org/10.1016/j.cpc.2013.09.018>
- [4] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, and O. Beckstein, MDAnalysis: A toolkit for the analysis of molecular dynamics simulations, *J. Comput. Chem.* **32**(10), 2319–2327 (2011). <https://doi.org/10.1002/jcc.21787>
- [5] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, *et al.*, SciPy 1.0: fundamental algorithms for scientific computing in Python, *Nat. Methods* **17**(3), 261–272 (2020). <https://doi.org/10.1038/s41592-019-0686-2>
- [6] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, *et al.*, Array programming with NumPy, *Nature* **585**(7825), 357–362 (2020). <https://doi.org/10.1038/s41586-020-2649-2>
- [7] E. Tiesinga, P. J. Mohr, D. B. Newell, and B. N. Taylor, CODATA recommended values of the fundamental physical constants: 2018, *Rev. Mod. Phys.* **93**(2), 025010 (2021). <https://doi.org/10.1103/RevModPhys.93.025010>
- [8] B. L. Welch, The generalization of ‘Student’s’ problem when several different population variances are involved, *Biometrika* **34**(1–2), 28–35 (1947). <https://doi.org/10.1093/biomet/34.1-2.28>

- [9] R. B. D’Agostino, Transformation to normality of the null distribution of g_1 , *Biometrika* **57**(3), 679–681 (1970). <https://doi.org/10.1093/biomet/57.3.679>
- [10] R. B. D’Agostino and E. S. Pearson, Tests for departure from normality. Empirical results for the distributions of b_2 and $\sqrt{b_1}$, *Biometrika* **60**(3), 613–622 (1973). <https://doi.org/10.1093/biomet/60.3.613>
- [11] F. J. Anscombe and W. J. Glynn, Distribution of the kurtosis statistic b_2 for normal samples, *Biometrika* **70**(1), 227–234 (1983). <https://doi.org/10.1093/biomet/70.1.227>
- [12] M. B. Brown and A. B. Forsythe, Robust tests for the equality of variances, *J. Am. Stat. Assoc.* **69**(346), 364–367 (1974). <https://doi.org/10.1080/01621459.1974.10482955>
- [13] S. Holm, A simple sequentially rejective multiple test procedure, *Scand. J. Stat.* **6**(2), 65–70 (1979).
- [14] B. Iglewicz and D. C. Hoaglin, *How to Detect and Handle Outliers*, ASQC Quality Press, Milwaukee, WI (1993).
- [15] F. E. Grubbs, Procedures for detecting outlying observations in samples, *Technometrics* **11**(1), 1–21 (1969). <https://doi.org/10.1080/00401706.1969.10490657>
- [16] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed., Cambridge University Press, Cambridge (2007).
- [17] N. Rego and D. Koes, 3Dmol.js: molecular visualization with WebGL, *Bioinformatics* **31**(8), 1322–1324 (2015). <https://doi.org/10.1093/bioinformatics/btu829>
- [18] O. M. Daramola, STEMKit: A client-side toolkit for computational chemistry and scientific data analysis (v0.1.1), Zenodo (2026). <https://doi.org/10.5281/zenodo.21543112>